

Smart Learning Management System (LMS) Backend

Production-Style RESTful Backend Architecture & Implementation Blueprint

Prepared for: User Review

Status: Completed & Compiled Successfully

Date: May 20, 2026

Version: 1.0.0-SNAPSHOT

Powered by: Spring Boot 3.1.5, Spring Security, JWT, MySQL & OpenPDF

1. Project Goal & Objectives

The Smart Learning Management System (LMS) Backend is an enterprise-grade backend infrastructure designed for online educational platforms. The architecture enables frictionless course administration, comprehensive enrollment control, automated tracking of student completion stages, secure role-based access mechanisms, and professional analytics reporting.

Objectives completed:

- Decentralized course management allowing instructors to perform full CRUD on courses.
- Double-enrollment prevention guarding the transaction layer against duplicate student enrollments.
- Automated progress tracking compiling completion percentages across courses.
- Custom PDF report compilation containing scores, remarks, and dynamic achievements.
- Multi-threaded SMTP mail notification providing immediate asynchronous PDF dispatch.

2. System Architecture & Tech Stack

The backend leverages the industry-standard Spring Boot framework following a strict layered architecture pattern (Controller -> Service -> Repository) to ensure modular development and easy scaling.

Technology Component	Version & Specific Role in LMS Backend
Java Platform	Java 17 (LTS) - Provides support for records, enhanced switch patterns, and modern performance improvements.
Spring Boot Framework	Spring Boot 3.1.5 - Provides auto-configurations for Web REST MVC, dependency injection, and JPA Hibernate ORM mapping.
Spring Security & JWT	Spring Security 6.0 & JSON Web Tokens (JJWT) - Handles stateless session filtering and robust Role-Based Access Control (RBAC).
Database Layer	MySQL 8.0 & Hibernate - Stores relational mappings for Users, Courses, and Enrollments with automated schema updates.
PDF Generation Engine	OpenPDF 1.3.30 (iText Open Source) - Compiles student records into beautiful, structured PDF canvases dynamically.
SMTP Email Dispatch	Spring Mail (JavaMailSender) - Constructs multipart MIME payloads to send PDF reports asynchronously via SMTP.
Caching Engine	Spring Starter Cache (ConcurrentMapCache) - Caches heavy catalog queries to reduce DB hits.
Asynchronous Execution	ThreadPoolTaskExecutor (@Async) - Offloads SMTP tasks into separate background threads to preserve HTTP thread pools.

3. Relational Schema & Database Design

The backend database is designed with normalized entities. The primary connection between students and courses is established via the 'enrollments' table, representing a many-to-many relationship with supplemental attributes.

3.1 User Entity (users table)

Stores system credentials and credentials metadata. The email is uniquely indexed.

- id: BIGINT (Primary Key, GenerationType.IDENTITY)
- name: VARCHAR(100) - Full name
- email: VARCHAR(100) (Unique Index)
- password: VARCHAR(120) - Hash coded using BCrypt Strength 10
- role: VARCHAR(50) - Enumerated String (ADMIN, INSTRUCTOR, STUDENT)

3.2 Course Entity (courses table)

Represents individual courses on the platform.

- id: BIGINT (Primary Key, GenerationType.IDENTITY)
- title: VARCHAR(150) - Title of the course
- description: VARCHAR(1000) - Summary outline
- category: VARCHAR(100) - Classification category
- instructor_id: BIGINT - Foreign Key pointing to users table
- difficulty_level: VARCHAR(50) - Beginner, Intermediate, Advanced

3.3 Enrollment Entity (enrollments table)

Acts as the relation join. The combination of student_id and course_id is uniquely indexed.

- id: BIGINT (Primary Key, GenerationType.IDENTITY)
- student_id: BIGINT - Foreign Key pointing to users table (role must be STUDENT)
- course_id: BIGINT - Foreign Key pointing to courses table
- progress: DOUBLE - Completion percentage from 0.0 to 100.0
- status: VARCHAR(50) - Enumerated String (ACTIVE, COMPLETED, DROPPED)
- enrolled_date: TIMESTAMP - Registration time stamp

4. Complete API & Role Permission Matrix

API Endpoint URL	Method	Authorized Roles	Action Summary
/api/auth/register	POST	Permit All	Registers a new user, hashes password with BCrypt.
/api/auth/login	POST	Permit All	Authenticates credentials and returns a secure JWT.
/api/courses	GET	Permit All	Lists all courses with paginated search, filter & caching.
/api/courses/{id}	GET	Permit All	Fetches single course details from cache or db.
/api/courses	POST	INSTRUCTOR, ADMIN	Creates a new course catalog record. Evicts caches.
/api/courses/{id}	PUT	INSTRUCTOR, ADMIN	Modifies course parameters. Evicts caches.
/api/courses/{id}	DELETE	INSTRUCTOR, ADMIN	Permanently deletes course. Evicts caches.
/api/enroll/{courseId}	POST	STUDENT	Enrolls active student in course. Blocks duplicates.
/api/enroll/{courseId}	DELETE	STUDENT	Unenrolls student and deletes relation record.
/api/student/enrollments	GET	STUDENT	Returns active course subscription list of current student.
/api/student/enrollments/{courseId}/progress	PUT	STUDENT, INSTRUCTOR	Updates course progress. If 100%, changes status to COMPLETED.
/api/student/{id}/progress	GET	STUDENT, INSTRUCTOR	Calculates score, completed courses, and averages.
/api/student/{id}/report/pdf	GET	STUDENT, INSTRUCTOR	Generates and streams a custom progress PDF download.
/api/student/{id}/report/email	GET	STUDENT, INSTRUCTOR	Generates PDF and sends via multi-threaded async email.
/api/users	GET/POST/PUT/DELETE	ADMIN	Full user profile controls for administrative staff.

5. In-Depth Project Walkthrough & Implementation Details

5.1 JWT Stateless Security Implementation

Authentication is implemented using stateless token filtering. Upon successful authentication at `/api/auth/login`, the `JwtTokenProvider` generates a secure JWT compact string, signing it with an HMAC SHA-512 secret key configured in `application.properties`. The payload encapsulates standard metadata, User ID, and the parsed Role. Subsequent calls pass this key inside the `'Authorization: Bearer'` header. The `JwtAuthenticationFilter` intercepts the requests, extracts the JWT, decodes the claims, retrieves standard credentials via `CustomUserDetailsService`, and builds a `UsernamePasswordAuthenticationToken` which is stored inside the Spring `SecurityContextHolder`.

5.2 Student Progress Analytics & Performance Score

Academic metrics are dynamically aggregated by `ProgressService`. Every enrollment records progress from 0.0 to 100.0. When a student hits 100.0, the status transitions automatically from `ACTIVE` to `COMPLETED`. The Performance Score is computed in real-time on the service layer using the formula:

$$\text{Score} = (\text{Completed Courses} * 50) + (\text{Average Progress} * 0.5)$$

All analytics are aggregated inside a clean, unexposed `ProgressDto` returned to the controller.

5.3 Dynamic PDF Compilation (OpenPDF)

The `ReportService` utilizes the OpenPDF toolkit to construct vector document grids. It dynamically draws title paragraphs, separation dividers, metadata profile lists, and a catalog matrix documenting course-wise progress and status. It also dynamically evaluates student metrics and assigns special achievements:

- [GOLD BADGE] - unlocked if performance score reaches or exceeds 100.
- [ELITE GRADUATE] - unlocked if completed courses count is at least 2.
- [HIGH ACHIEVER] - unlocked if student maintains a progress average above 75%.

Remarks are calculated contextually, pointing out areas of success or recommending extra focus.

5.4 Asynchronous Email Notifications (@Async)

Offloading high-latency networking operations is critical for UI performance. When a user triggers `/api/student/{id}/report/email`, the report is generated, and the bytes are forwarded to `EmailService`'s `sendReportEmailWithAttachment` method. This method is annotated with `@Async('mailExecutor')` and is routed to a custom `ThreadPoolTaskExecutor` thread configured in `AsyncConfig`. The calling HTTP thread receives an immediate 200 OK success payload notifying that the email process has started in the background, while the SMTP delivery completes safely without blocking user interaction.

6. Database Seeding & Verification Guides

To enable immediate test cycles, the main LmsBackendApplication integrates a CommandLineRunner bean which checks the database state upon boot. If the databases are blank, it creates:

1. ADMIN Profile: admin@lms.com / admin123
2. INSTRUCTOR Profile: instructor@lms.com / instructor123 (Dr. Sarah Jenkins)
3. STUDENT Profile: student@lms.com / student123 (Alex Carter)
4. Three core courses taught by Dr. Jenkins.

Steps for testing APIs using curl:

Step 1: Perform login as student:

```
curl -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" -d
{"email":"student@lms.com","password":"student123"}
```

Step 2: Copy the 'accessToken' from response.

Step 3: Access student progress metrics (replacing <TOKEN> and <ID>):

```
curl -X GET http://localhost:8080/api/student/<ID>/progress -H "Authorization: Bearer <TOKEN>"
```

Step 4: Download PDF progress report:

```
curl -X GET http://localhost:8080/api/student/<ID>/report/pdf -H "Authorization: Bearer <TOKEN>" --output
report.pdf
```

Step 5: Send PDF report asynchronously via email:

```
curl -X GET http://localhost:8080/api/student/<ID>/report/email -H "Authorization: Bearer <TOKEN>"
```

This concludes the formal technical blueprint for the Smart LMS Backend Project.